Isorite agent integration: three routes compared

Nov 4, 2026 · @Lachlan Stewart

Overview

Isorite can reach an agent three ways, and they stack: all three can ship together, and the later two reuse the first one's tools.

- **1. MCP only.** The user runs their own harness (Claude Code, Codex, OpenCode) and isorite exposes the notes as MCP tools. Isorite only sees what the agent does through those tools.
- **2. MCP + ACP.** Isorite launches the user's agent through an ACP adapter and drives it from the web UI. It can attach context to each prompt and show the whole session.
- **3. Embedded pi.** Isorite runs the pi agent inside its own Node process through pi's SDK, so it owns the loop, the tools and the permissions.

Two things are shared. The same tool code can back the MCP server and the in-process pi tools. Using ACP's session-update shape as isorite's internal event model lets route 3 plug into the same UI as route 2.

Pi can also be driven as an ACP agent through a third-party adapter (`pi-acp`), so route 2 covers pi users too. Route 3 is distinct only because it embeds pi through the SDK.

Comparison

Only route 3 can enforce permissions and steer mid-turn; route 2 is the one that sends highlights to the user's own agent.

Feature	1. MCP only	2. MCP + ACP	3. Embedded pi
Agent reads and writes notes	Yes, via MCP	Yes, via MCP	Yes, in-process (MCP optional)
Send a highlight to the agent	No; only a pull tool the agent must call	Yes; embedded resource, or text fallback	Yes; direct
Session visible in isorite	Only calls to isorite's tools	Full stream	Full stream

Feature	1. MCP only	2. MCP + ACP	3. Embedded pi
Runs in the user's own harness UI	Yes	No; separate SDK-driven session	No
User's MCP servers, skills, CLAUDE.md	Native	Probably; unverified per adapter, depends on launch environment	No; pi's own system, MCP via extension
Slash commands	All of the harness's	Whatever the adapter advertises	Ones you build, plus pi's
Enforce edit/file/command switches	No	Partial; the agent's own tools can bypass	Yes; `tool_call` hook, fail-closed
Steer mid-turn	No	Cancel only, as far as I found	Yes; `steer()` and `followUp()` queue input
Control context and compaction	No	No	Yes
Agent choice	Any MCP-capable agent	Any agent with an ACP adapter	Pi's providers (many), API key or subscription login
Upkeep for you	Low	Medium to high; adapters and agent versions vary	High; you own a harness, pi is pre-1.0 and renamed its package

Pros and cons

Route 1 is cheapest and keeps the user's setup intact, route 2 adds the interaction you want, and route 3 adds control at the cost of owning an agent.

1. MCP only

- **Pro:** users keep their full harness, skills and habits.
- **Pro:** cheapest to build and works with any MCP-capable agent.
- **Con:** no push, so a highlight reaches the agent only if it calls a tool.
- **Con:** isorite sees only calls to its own tools and cannot enforce its switches on the agent's other tools.

2. MCP + ACP

- **Pro:** highlight-to-agent, with the full session shown in the web UI.
- **Pro:** the user's own agent of choice, probably with their existing setup.
- **Pro:** the same wiring lets ACP front ends such as Zed share the session.
- **Con:** a testing matrix across adapters and agent versions.
- **Con:** it is a separate SDK-driven session, not the user's terminal session, and some harness features may be missing.

3. Embedded pi

- **Pro:** enforcement through a fail-closed `tool_call` hook, and mid-turn steering.
- **Pro:** control over context and compaction, and direct calls into the notes.
- **Pro:** TypeScript like the rest of the repo, and no fork needed.
- **Con:** isorite owns an agent again, which version 0.13 removed.
- **Con:** users lose their existing setup, and pi's API is still moving.

Use cases

Pick the route by who the user is and how much they need guaranteed.

- **Route 1:** developers with a deep harness setup who mainly want the agent to read and write their notes, for example while coding.
- **Route 2:** users who like their agent but want the Atlas, highlight-to-agent and the session in one window.
- **Route 3:** newcomers with no harness, and anyone who needs guarantees, such as an enforced read-only mode or a classroom where the agent must not touch project files.

To verify

Five claims above rest on documentation I could not confirm against the actual adapters, so each is worth a short test.

- [ ] Which settings sources, MCP config and environment the Claude ACP adapter loads in an isorite session.
- [ ] Whether that adapter advertises the user's own slash commands from `.claude/commands`.
- [ ] Whether each adapter declares embedded-context support, which decides between an embedded resource and the text fallback for highlights.
- [ ] Whether a terminal Claude Code session can be resumed through ACP, or the reverse.

- [ ] Whether pi's `steer()` interrupts the current run or only queues input after the current tool calls; two sources describe it differently.

A one-day prototype settles the pi question: embed pi, add an extension that reads isorite's current selection, and enforce the edit switch with a `tool_call` hook.

Sources

These come from search results and, for isorite, its README; apart from the README I did not open the pages in full, so treat details as unconfirmed until tested.

- Isorite README
- ACP content blocks
- CodeCompanion ACP notes
- ACP session and commands (ExMCP guide)
- acpx agent mapping, including pi-acp
- Claude Agent SDK settings sources
- Pi SDK docs
- Pi SDK reference
- Pi extension API notes
- Pi design and omissions